

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего
образования

«КАЗАНСКИЙ (ПРИВОЛЖСКИЙ) ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Набережночелнинский институт

ОТЧЁТ

по дисциплине «Кроссплатформенное программное обеспечение»

Тема проекта:

«Система мониторинга пожарной безопасности учебного корпуса»

Выполнил: студент группы 2232122

Шайхетдинов Д.Ш.

Набережные Челны, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. ТРЕБОВАНИЯ К ПРОЕКТУ И ИХ ВЫПОЛНЕНИЕ	4
2. ИЗУЧЕННЫЕ ТЕХНОЛОГИИ И ИНСТРУМЕНТЫ	6
2.1. Java и Maven.....	6
2.2. HTTP, REST и Spring Web.....	6
2.3. DI/IOC в Spring	6
2.4. Spring Data JPA и PostgreSQL	7
2.5. Spring Security, JWT и BCrypt	7
2.6. Swagger/OpenAPI.....	7
2.7. Работа с файлами, CSV, PDF и XLSX.....	7
2.8. Docker, логирование и Telegram	8
3. ОПИСАНИЕ РАЗРАБОТАННОЙ СИСТЕМЫ.....	9
4. АРХИТЕКТУРА ПРИЛОЖЕНИЯ.....	10
5. СТРУКТУРА БАЗЫ ДАННЫХ	12
6. БЕЗОПАСНОСТЬ, РОЛИ И ПРАВА ДОСТУПА.....	14
7. РЕАЛИЗАЦИЯ КЛЮЧЕВОЙ ФУНКЦИОНАЛЬНОСТИ	15
7.1. CRUD-операции	15
7.2. Автоматическое создание тревоги	15
7.3. Telegram-уведомления	15
7.4. Загрузка файлов.....	16
7.5. Импорт CSV	16
7.6. PDF и XLSX отчёты	16
8. ТЕСТИРОВАНИЕ, ЗАПУСК И ДЕМОНСТРАЦИЯ.....	17
9. ПРАКТИЧЕСКОЕ ПРИМЕНЕНИЕ И РАЗВИТИЕ ПРОЕКТА.....	18
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	20

ВВЕДЕНИЕ

В рамках дисциплины «Кроссплатформенное программное обеспечение» был разработан backend REST API для системы мониторинга пожарной безопасности учебного корпуса. Проект предназначен для учёта корпусов, помещений, датчиков, показаний, тревог и инцидентов, связанных с обеспечением пожарной безопасности образовательной организации.

Основная идея системы заключается в том, что при поступлении показания от датчика выполняется проверка значения относительно установленного порога. Если значение превышает допустимый предел, приложение автоматически создаёт тревогу, фиксирует событие в базе данных, записывает лог и отправляет уведомление через Telegram-бота.

Проект выполнен на языке Java с использованием Spring Boot и может запускаться на разных операционных системах за счёт JVM. В работе применены технологии, изученные в рамках курса: Maven, REST API, Spring Web, Spring Data JPA, PostgreSQL, Spring Security, JWT, Swagger/OpenAPI, Docker Compose, логирование, работа с файлами, импорт CSV, формирование PDF/XLSX-отчётов и интеграция с Telegram.

Цель отчёта — описать изученные на курсе технологии, показать, как они применены в разработанном проекте, а также зафиксировать возможности дальнейшего использования полученных знаний.

1. ТРЕБОВАНИЯ К ПРОЕКТУ И ИХ ВЫПОЛНЕНИЕ

Финальный проект был разработан с учётом требований к сдаче по дисциплине. Основное внимание уделялось не только CRUD-операциям, но и реальной прикладной логике, безопасности, документации API, работе с файлами, отчётами, Telegram-уведомлениями и истории разработки в GitHub.

Требование	Реализация в проекте
Полный CRUD	Реализованы операции создания, чтения, изменения и удаления для корпусов, помещений, датчиков, пользователей и других сущностей.
Security	Реализована JWT-аутентификация, BCrypt-хэширование паролей, фильтр проверки токена и ограничение доступа к endpoint-ам.
Роли и permissions	Созданы роли ADMIN, DISPATCHER, TECHNICIAN, VIEWER и набор прав доступа. Доступ проверяется через @PreAuthorize.
Swagger	API документировано через springdoc-openapi. Swagger UI доступен по адресу /swagger-ui/index.html.
Логирование	Ключевые бизнес-события, ошибки, тревоги и действия сервисов логируются через SLF4J/Logback.
Telegram-логи	При создании тревоги формируется сообщение в Telegram, результат отправки сохраняется в TelegramLog.
Загрузка файлов	К инциденту можно прикрепить фото. Проверяются MIME-тип и размер файла.
Чтение из файлов	Реализован импорт датчиков из CSV-файла с сохранением результата импорта.
Формирование отчётов	Формируются отчёты по тревогам в форматах PDF и XLSX.
Схемы БД	Подготовлены логическая и физическая схемы БД в документации проекта.
GitHub	Проект размещён в репозитории fire-safety-monitoring-api, история разработки содержит последовательные коммиты.

Требование	Реализация в проекте
Скриншоты и видео	В папке docs/screenshots сохранены скриншоты запуска на виртуальной машине, в docs/demo размещено видео демонстрации.

2. ИЗУЧЕННЫЕ ТЕХНОЛОГИИ И ИНСТРУМЕНТЫ

2.1. Java и Maven

Java была использована как основной язык разработки. Её преимущество для кроссплатформенного программного обеспечения заключается в выполнении байт-кода на виртуальной машине JVM. Благодаря этому приложение может запускаться в Windows, Linux и macOS без изменения исходного кода.

Maven применён для управления зависимостями, сборкой и жизненным циклом проекта. Файл pom.xml содержит зависимости Spring Boot, Spring Security, PostgreSQL Driver, JWT, Apache POI, OpenPDF, OpenCSV, Lombok, JUnit и Mockito. Запуск и сборка выполняются командами `mvn spring-boot:run`, `mvn clean test` и `mvn clean package`.

2.2. HTTP, REST и Spring Web

REST API построен на стандартных HTTP-методах: GET используется для получения данных, POST — для создания, PUT — для обновления, DELETE — для удаления. Такой подход позволяет использовать приложение через Swagger, Postman, curl или будущий веб/мобильный клиент.

Spring Web MVC обеспечивает обработку HTTP-запросов через контроллеры с аннотациями `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping` и `@DeleteMapping`. Для передачи данных применяются DTO-классы, а валидация выполняется с помощью `jakarta.validation`.

2.3. DI/IoC в Spring

Контейнер Spring управляет созданием объектов и внедрением зависимостей. В проекте сервисы получают репозитории и другие сервисы через конструктор, что делает код слабосвязанным и удобным для тестирования. Такой подход соответствует принципам инверсии управления и `dependency injection`.

```
@Service
@RequiredArgsConstructor
public class SensorReadingService {
```

```

private          final          SensorRepository          sensorRepository;
private          final          AlertService              alertService;
}

```

2.4. Spring Data JPA и PostgreSQL

Для хранения данных используется PostgreSQL. Доступ к данным реализован через Spring Data JPA. Сущности описаны с помощью аннотаций `@Entity`, `@Table`, `@Id`, `@ManyToOne`, `@OneToMany` и `@ManyToMany`. Репозитории наследуются от `JpaRepository`, что позволяет не писать типовые SQL-запросы вручную.

В проекте используются транзакции, что важно при создании показания датчика и последующем создании тревоги. Если операция завершается ошибкой, данные не должны оказаться в неконсистентном состоянии.

2.5. Spring Security, JWT и BCrypt

Безопасность реализована через Spring Security и JWT. При входе пользователь отправляет логин и пароль, после чего получает токен. Далее токен передаётся в заголовке `Authorization: Bearer`. Пароли хранятся не в открытом виде, а в виде BCrypt-хэша.

Проверка доступа выполняется через роли и конкретные права. Это позволяет ограничить действия пользователей: например, `VIEWER` может только просматривать данные, `TECHNICIAN` — работать с инцидентами, `DISPATCHER` — обрабатывать тревоги, `ADMIN` — управлять всей системой.

2.6. Swagger/OpenAPI

Swagger используется как интерактивная документация API. Он позволяет просматривать все endpoint-ы, отправлять тестовые запросы, передавать JWT-токен и проверять работу системы без отдельного фронтенда.

2.7. Работа с файлами, CSV, PDF и XLSX

Работа с файлами реализована в нескольких направлениях: загрузка фото к инциденту через `multipart/form-data`, импорт датчиков из CSV-файла, формирование

отчётов по тревогам в PDF и XLSX. Для XLSX используется Apache POI, для PDF — OpenPDF, для импорта CSV — OpenCSV.

2.8. Docker, логирование и Telegram

PostgreSQL запускается через docker-compose.yml, что упрощает подготовку окружения. Логирование реализовано через SLF4J/Logback. Telegram Bot API используется для отправки уведомлений о тревогах. При отсутствии токена Telegram приложение не падает, а записывает предупреждение в лог.

3. ОПИСАНИЕ РАЗРАБОТАННОЙ СИСТЕМЫ

Разработанная система предназначена для автоматизации контроля пожарной безопасности учебного корпуса. Она позволяет учитывать корпуса, помещения, датчики, показания датчиков, тревоги, инциденты, фотографии к инцидентам, Telegram-уведомления, импорт данных и генерацию отчётов.

В типовом сценарии диспетчер или внешняя система добавляет показание датчика. Система получает датчик из базы данных, сравнивает значение показания с пороговым значением и, если порог превышен, создаёт тревогу. Далее тревога может быть переведена в статус IN_PROGRESS, RESOLVED или FALSE_ALARM. При необходимости на основе тревоги создаётся инцидент, к которому техник может приложить фото.

Функция	Описание
Учёт корпусов и помещений	Позволяет описать структуру учебного корпуса и расположение датчиков.
Учёт датчиков	Хранит инвентарный номер, тип, статус, помещение, дату установки и пороговое значение.
Приём показаний	Фиксирует измеренные значения и время измерения.
Автоматические тревоги	При превышении порога создаётся запись Alert и отправляется уведомление.
Инциденты	Позволяют назначить ответственного сотрудника и контролировать обработку события.
Фото инцидентов	Поддерживается загрузка изображений к инциденту.
Импорт CSV	Позволяет быстро загрузить список датчиков.
Отчёты	Формируются PDF/XLSX отчёты за выбранный период.

4. АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Проект построен по классической многослойной архитектуре Spring Boot: controller → service → repository → entity. Контроллеры принимают HTTP-запросы, сервисы содержат бизнес-логику, репозитории отвечают за доступ к базе данных, а сущности описывают структуру таблиц.

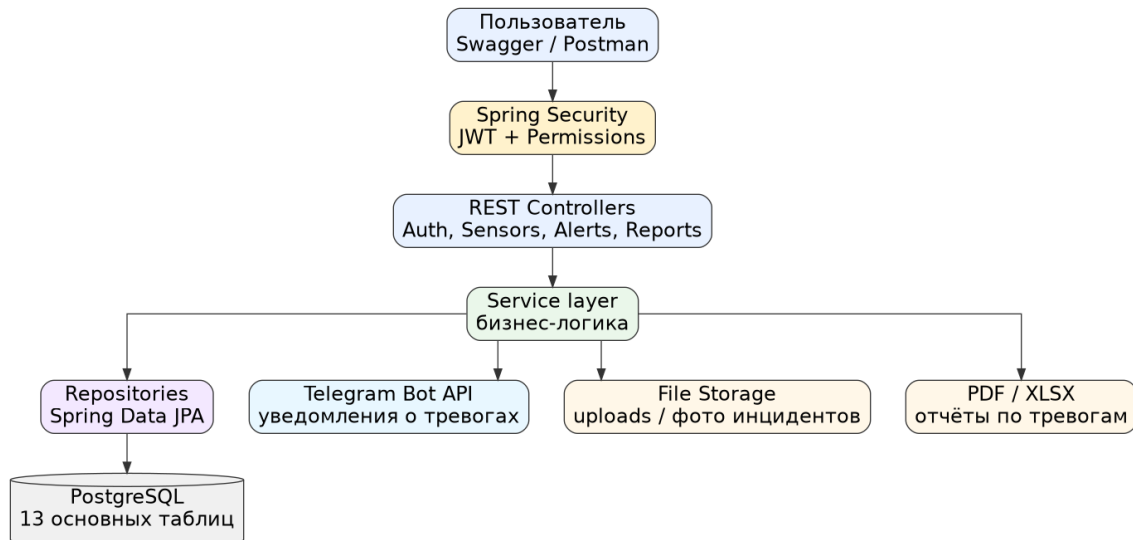


Рисунок 1 – Общая архитектура приложения

Пакет	Назначение
config	Конфигурация Swagger, инициализация тестовых данных.
controller	REST-контроллеры для обработки HTTP-запросов.
dto	Request/Response DTO для обмена данными через API.
entity	JPA-сущности, соответствующие таблицам БД.
enums	Перечисления: статусы, типы датчиков, типы отчётов.
exception	Кастомные исключения и глобальный обработчик ошибок.
mapper	Преобразование сущностей в DTO.
repository	Интерфейсы Spring Data JPA.
security	JWT, фильтр авторизации, UserDetailsService, SecurityConfig.
service	Основная бизнес-логика приложения.

В проекте реализованы контроллеры AuthController, UserController, BuildingController, RoomController, SensorController, ReadingController,

AlertController, IncidentController, FileController, ImportController, ReportController и TelegramController. Такое разделение делает API понятным и удобным для демонстрации.

5. СТРУКТУРА БАЗЫ ДАННЫХ

База данных построена вокруг доменных сущностей системы пожарного мониторинга. Основные связи: корпус содержит помещения, помещение содержит датчики, датчик формирует показания, превышение показаний создаёт тревогу, тревога может быть связана с инцидентом, к инциденту прикрепляются фотографии.

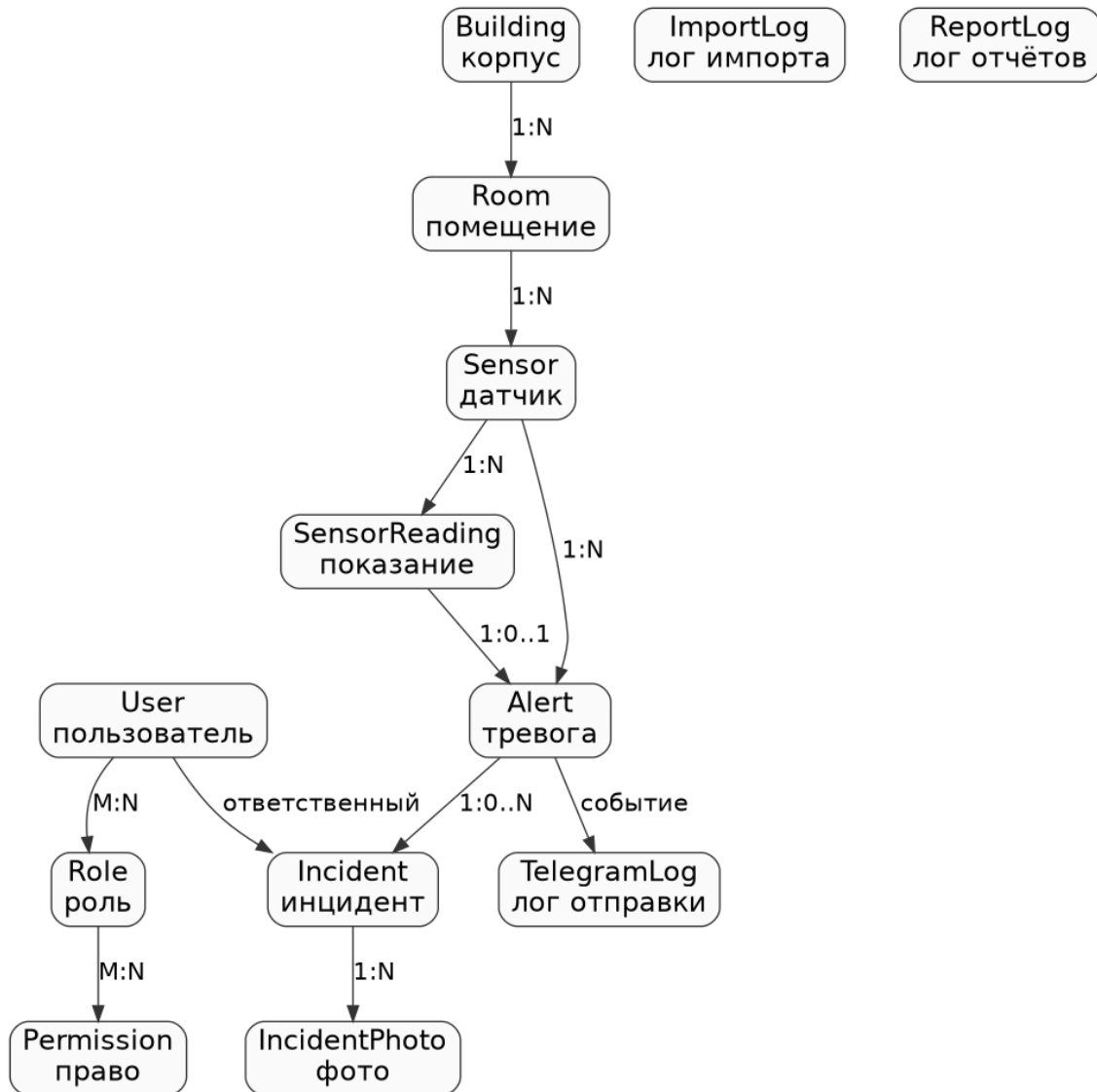


Рисунок 2 – Логическая схема базы данных

Таблица	Назначение
users	Пользователи системы, логины, пароли, email, активность.

Таблица	Назначение
roles	Роли пользователей: ADMIN, DISPATCHER, TECHNICIAN, VIEWER.
permissions	Права доступа, используемые в @PreAuthorize.
buildings	Учебные корпуса.
rooms	Помещения внутри корпусов.
sensors	Пожарные датчики и их параметры.
sensor_readings	Показания датчиков.
alerts	Тревоги, созданные при превышении порога.
incidents	Инциденты, связанные с тревогами.
incident_photos	Загруженные фото к инцидентам.
telegram_logs	Результаты отправки Telegram-уведомлений.
import_logs	Результаты импорта данных из CSV.
report_logs	Факты генерации отчётов.

6. БЕЗОПАСНОСТЬ, РОЛИ И ПРАВА ДОСТУПА

В системе реализована ролевая модель с granular permissions. При запуске DataInitializer создаёт необходимые права, роли и тестовых пользователей. Это позволяет сразу проверить работу авторизации и разграничения доступа.

Роль	Права и назначение
ADMIN	Полный доступ ко всем функциям системы.
DISPATCHER	Просмотр датчиков, создание показаний, обработка тревог, формирование отчётов.
TECHNICIAN	Просмотр и обработка инцидентов, загрузка фотографий.
VIEWER	Просмотр справочной информации без права изменения данных.

Пример проверки доступа в коде:

```
@PreAuthorize("hasAuthority('SENSOR_CREATE')")
@PostMapping
public ResponseEntity<SensorResponse> create(@Valid @RequestBody
SensorRequest request) {
    return ResponseEntity.ok(sensorService.create(request));
}
```

Таким образом, даже если пользователь знает адрес endpoint-a, он не сможет выполнить действие без соответствующего права. Это соответствует требованию использовать permissions и @PreAuthorize.

7. РЕАЛИЗАЦИЯ КЛЮЧЕВОЙ ФУНКЦИОНАЛЬНОСТИ

7.1. CRUD-операции

CRUD-операции реализованы для основных справочников и рабочих сущностей. Например, для датчиков доступны получение списка, получение по идентификатору, создание, обновление и удаление. Аналогичный подход реализован для корпусов, помещений и пользователей.

Операция	Пример endpoint-а
Create	POST /api/sensors — создание датчика.
Read	GET /api/sensors и GET /api/sensors/{id} — просмотр датчиков.
Update	PUT /api/sensors/{id} — изменение параметров датчика.
Delete	DELETE /api/sensors/{id} — удаление датчика.

7.2. Автоматическое создание тревоги

При добавлении показания датчика сервис `SensorReadingService` сравнивает значение `value` с пороговым значением датчика `thresholdValue`. Если значение выше порога, создаётся объект `Alert`. Такой сценарий показывает не просто CRUD, а реальную бизнес-логику системы.

```
POST /api/readings
{
  "sensorId": 1,
  "value": 99.9
}
```

7.3. Telegram-уведомления

После создания тревоги вызывается `TelegramService`. Он формирует текст уведомления и отправляет его через Telegram Bot API. Если токен или `chat_id` не заданы, система продолжает работать, а в лог записывается предупреждение. Это важно для безопасного запуска проекта без секретов в репозитории.

7.4. Загрузка файлов

Для инцидента реализована загрузка фотографии через multipart/form-data. FileStorageService проверяет MIME-тип, размер файла, сохраняет файл с уникальным именем и записывает метаданные в таблицу incident_photos. Это демонстрирует работу со статическими файлами и безопасность загрузки.

7.5. Импорт CSV

CsvImportService читает CSV-файл с датчиками. Формат файла: inventoryNumber,type,status,roomId,thresholdValue. После импорта сохраняется ImportLog с количеством успешно импортированных и ошибочных записей.

7.6. PDF и XLSX отчёты

ReportService формирует отчёты по тревогам за указанный период. PDF-отчёт подходит для передачи руководству, XLSX-отчёт — для анализа данных в табличном процессоре. Каждый факт формирования отчёта сохраняется в ReportLog.

8. ТЕСТИРОВАНИЕ, ЗАПУСК И ДЕМОНСТРАЦИЯ

Для проверки проекта добавлены тесты на сервисный слой. В тестах используются JUnit 5 и Mockito. Проверяются логика авторизации, создание датчиков, создание тревоги при превышении порога, импорт CSV и smoke-проверка генерации отчётов.

Тест	Что проверяет
AuthServiceTest	Проверка логики входа пользователя и выдачи JWT.
SensorServiceTest	Проверка создания датчика и обработки ошибок.
SensorReadingServiceTest	Проверка создания показания и автоматической тревоги.
CsvImportServiceTest	Проверка импорта датчиков из CSV.

Проект запускается следующими командами:

```
docker compose up -d
mvn spring-boot:run
```

После запуска Swagger UI доступен по адресу <http://localhost:8080/swagger-ui/index.html>. Тестовый пользователь администратора: admin / admin123. В папке docs/screenshots размещены скриншоты запуска на виртуальной машине, а в docs/demo — видео демонстрации работы функционала.

История работы в GitHub содержит последовательные коммиты, отражающие этапы разработки: создание проекта, добавление сущностей и репозитория, настройка безопасности, реализация сервисов, контроллеров, Swagger, тестов, документации, скриншотов и демонстрационного видео.

Пример коммитов:

```
Initial Spring Boot project setup
Create domain entities and repositories
Configure Spring Security with JWT authentication
Add CRUD REST controllers with role-based access control
Add unit tests, documentation and database schemas
docs: add VM deployment screenshots and demo video
```

9. ПРАКТИЧЕСКОЕ ПРИМЕНЕНИЕ И РАЗВИТИЕ ПРОЕКТА

Полученные знания можно применять при разработке корпоративных информационных систем, сервисов мониторинга, систем оповещения и учёта событий. Проект пожарного мониторинга может быть расширен до реальной системы безопасности при подключении физических датчиков, IoT-шлюзов и брокеров сообщений.

Направление развития	Описание
Интеграция с IoT	Подключение реальных датчиков через MQTT, HTTP или промышленные шлюзы.
Kafka	Асинхронная обработка большого потока показаний и тревог.
Prometheus/Grafana	Мониторинг доступности, нагрузки и количества тревог.
Frontend или мобильное приложение	REST API можно использовать как backend для веб/мобильного клиента.
Микросервисная архитектура	Выделение Alert Service, Notification Service и Report Service.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта была разработана система мониторинга пожарной безопасности учебного корпуса на базе Java 17 и Spring Boot. Проект демонстрирует основные темы курса «Кроссплатформенное программное обеспечение»: кроссплатформенную разработку, REST API, работу с базой данных, DI/IoC, Spring Security, JWT, разграничение доступа, логирование, документацию API, контейнеризацию, работу с файлами, формирование отчётов и интеграцию с внешним сервисом Telegram.

Система не ограничивается простыми CRUD-операциями: в ней реализован прикладной сценарий обработки показаний датчиков, автоматического создания тревог, отправки уведомлений, ведения инцидентов и формирования отчётности. Это делает проект близким к реальной рабочей системе.

В результате выполнения проекта были закреплены навыки проектирования backend-приложений, построения многослойной архитектуры, настройки безопасности и документирования программного решения. Полученные знания могут быть применены в дальнейшей разработке веб-сервисов, корпоративных систем и сервисов мониторинга.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Boot Documentation. — URL: <https://docs.spring.io/spring-boot/> (дата обращения: 11.05.2026).
2. Spring Security Reference. — URL: <https://docs.spring.io/spring-security/> (дата обращения: 11.05.2026).
3. Spring Data JPA Reference Documentation. — URL: <https://docs.spring.io/spring-data/jpa/reference/> (дата обращения: 11.05.2026).
4. PostgreSQL Documentation. — URL: <https://www.postgresql.org/docs/> (дата обращения: 11.05.2026).
5. Docker Documentation. — URL: <https://docs.docker.com/> (дата обращения: 11.05.2026).
6. springdoc-openapi Documentation. — URL: <https://springdoc.org/> (дата обращения: 11.05.2026).
7. Apache POI Documentation. — URL: <https://poi.apache.org/> (дата обращения: 11.05.2026).
8. OpenPDF Project. — URL: <https://github.com/LibrePDF/OpenPDF> (дата обращения: 11.05.2026).
9. Telegram Bot API. — URL: <https://core.telegram.org/bots/api> (дата обращения: 11.05.2026).
10. JWT Library. — URL: <https://github.com/jwtkt/jjwt> (дата обращения: 11.05.2026).

ПРИЛОЖЕНИЕ А. СКРИНШОТЫ ЗАПУСКА И РАБОТЫ СИСТЕМЫ

```
yako@Ubuntu:~/Desktop/fire-safety-monitoring-api$ cat .env
TELEGRAM_BOT_TOKEN=8667596562:AAG6-2xnKWpgXU8v6dbtpnNgAPU1TYHBo_Q
TELEGRAM_CHAT_ID=9189151075
yako@Ubuntu:~/Desktop/fire-safety-monitoring-api$ sudo docker compose up -d
WARN[0000] /home/yako/Desktop/fire-safety-monitoring-api/docker-compose.yml: the
attribute 'version' is obsolete, it will be ignored, please remove it to avoid
potential confusion
[+] Running 0/12
 * postgres [          ] Pulling                               4.1s
 * ba2769f58dd8 Pulling fs layer                               0.0s
 * 6a0ac1617861 Pulling fs layer                               0.0s
 * 697a7e77ac2e Pulling fs layer                               0.0s
 * fae1993563ec Pulling fs layer                               0.0s
 * 013d59842ad1 Pulling fs layer                               0.0s
 * cb0bf5bcb56f Pulling fs layer                               0.0s
 * 9bf7646f4ff5 Pulling fs layer                               0.0s
 * 37a63010ee4d Pulling fs layer                               0.0s
 * 459be1018dcb Pulling fs layer                               0.0s
 * 523b9d9d6254 Pulling fs layer                               0.0s
 * 5206f96f8a8d Pulling fs layer                               0.0s
```

Рисунок А.1 – Проверка версий Java, Docker и Maven

```
2026-05-11 23:23:13 [main] WARN o.h.e.jdbc.spi.SqlExceptionHelper - SQL Warning Code: 0, SQLState: 00000
kipping
2026-05-11 23:23:13 [main] WARN o.h.e.jdbc.spi.SqlExceptionHelper - SQL Warning Code: 0, SQLState: 00000
2026-05-11 23:23:13 [main] WARN o.h.e.jdbc.spi.SqlExceptionHelper - constraint "uk_m13u6rafq5agfq1d14v9lsp4" of relation "sensors" does not exist, s
pping
2026-05-11 23:23:13 [main] WARN o.h.e.jdbc.spi.SqlExceptionHelper - constraint "uk_6dotkott2kjsp8vw4d0n25fb7" of relation "users" does not exist, s
pping
2026-05-11 23:23:13 [main] WARN o.h.e.jdbc.spi.SqlExceptionHelper - SQL Warning Code: 0, SQLState: 00000
2026-05-11 23:23:13 [main] WARN o.h.e.jdbc.spi.SqlExceptionHelper - constraint "uk_r43af9ap4edm43mntq01oddj6" of relation "users" does not exist, s
pping
2026-05-11 23:23:14 [main] INFO o.s.o.j.LocalContainerEntityManagerFactoryBean - Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-05-11 23:23:14 [main] DEBUG c.f.security.JwtAuthenticationFilter - Filter 'jwtAuthenticationFilter' configured for use
2026-05-11 23:23:15 [main] INFO o.s.d.j.r.query.QueryEnhancerFactory - Hibernate is in classpath; If applicable, HQL parser will be used.
2026-05-11 23:23:15 [main] INFO c.f.service.FileStorageService - Upload directory initialized: uploads
2026-05-11 23:23:16 [main] WARN o.s.b.a.o.j.JpaBaseConfiguration$JpaWebConfiguration - spring.jpa.open-in-view is enabled by default. Therefore, da
base queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-05-11 23:23:16 [main] INFO o.s.s.web.DefaultSecurityFilterChain - Will secure any request with [org.springframework.security.web.session.Disab
EncodeUrlFilter@7eee42fc, org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter@5967f0ff, org.springframework.secu
ty.web.context.SecurityContextHolderFilter@2d7e6c8c, org.springframework.security.web.header.HeaderWriterFilter@642aa8ea, org.springframework.web.fi
er.CorsFilter@580694ce, org.springframework.security.web.authentication.logout.LogoutFilter@60a6564c, com.firesafety.security.JwtAuthenticationFilter
2cf97875, org.springframework.security.web.savedrequest.RequestCacheAwareFilter@ced7e4a, org.springframework.security.web.servletapi.SecurityContext
lderAwareRequestFilter@af27e00, org.springframework.security.web.authentication.AnonymousAuthenticationFilter@5be521cd, org.springframework.security
eb.session.SessionManagementFilter@240fe338, org.springframework.security.web.access.ExceptionTranslationFilter@44c141ce, org.springframework.securi
.web.access.intercept.AuthorizationFilter@4740b5d]
2026-05-11 23:23:17 [main] INFO o.s.b.w.e.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path ''
2026-05-11 23:23:17 [main] INFO c.firesafety.FireSafetyApplication - Started FireSafetyApplication in 11.843 seconds (process running for 12.516)
2026-05-11 23:23:17 [main] INFO c.firesafety.config.DataInitializer - Initializing application data...
2026-05-11 23:23:18 [main] INFO c.firesafety.config.DataInitializer - Created 19 permissions
2026-05-11 23:23:18 [main] INFO c.firesafety.config.DataInitializer - Created 4 roles
2026-05-11 23:23:18 [main] INFO c.firesafety.config.DataInitializer - Created admin, dispatcher, technician users
2026-05-11 23:23:18 [main] INFO c.firesafety.config.DataInitializer - Created test building, 3 rooms, 5 sensors
2026-05-11 23:23:18 [main] INFO c.firesafety.config.DataInitializer - Data initialization complete.
```

Рисунок А.2 – Запуск PostgreSQL через Docker Compose

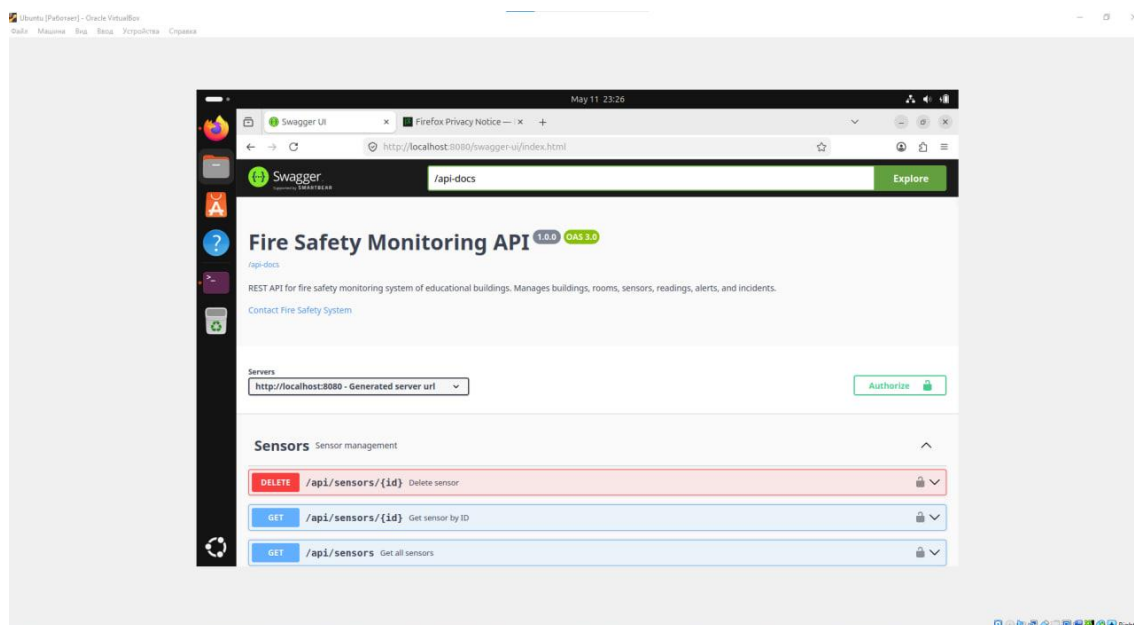


Рисунок А.3 – Запуск Spring Boot приложения

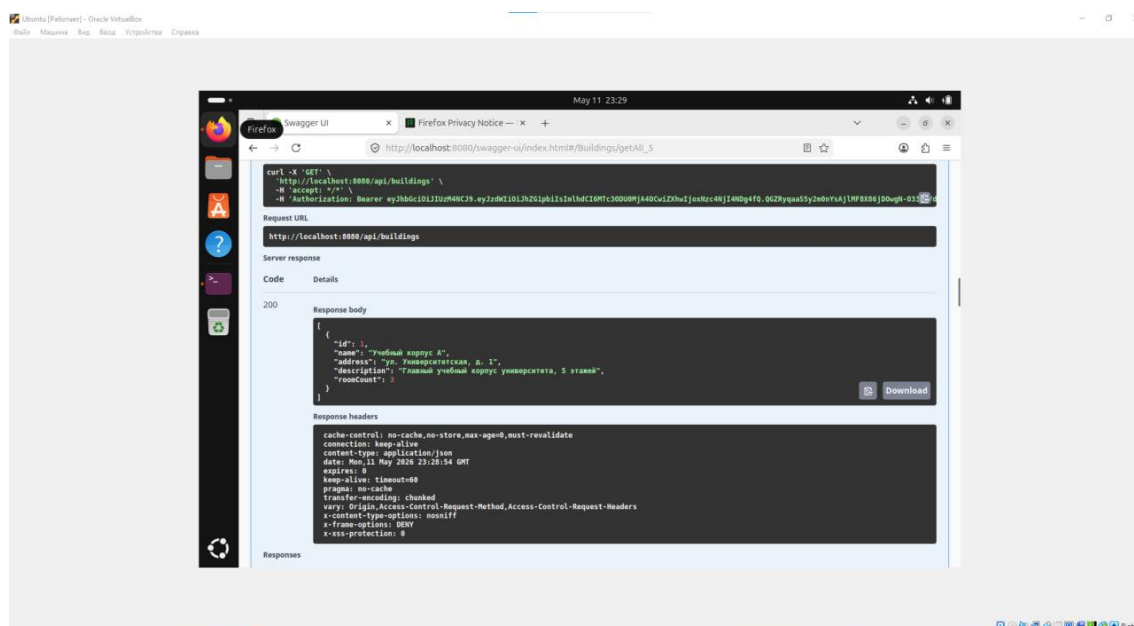


Рисунок А.4 – Swagger UI с документацией API

```
yako@Ubuntu:~/Desktop/fire-safety-monitoring-api$ java -version && docker --version && mvn -version
openjdk version "21.0.11-ea" 2026-04-21
OpenJDK Runtime Environment (build 21.0.11-ea+8-Ubuntu-1)
OpenJDK 64-Bit Server VM (build 21.0.11-ea+8-Ubuntu-1, mixed mode, sharing)
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
Apache Maven 3.9.12
Maven home: /usr/share/maven
Java version: 21.0.11-ea, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "7.0.0-15-generic", arch: "amd64", family: "unix"
yako@Ubuntu:~/Desktop/fire-safety-monitoring-api$
```

Рисунок А.5 – Пример успешного GET-запроса к API